

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Amos Harol Turnip - 5024241023

2026

1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi saat ini semakin mendorong penggunaan sistem tertanam (embedded system) dalam berbagai bidang kehidupan, seperti perangkat Internet of Things (IoT), sistem otomasi, peralatan rumah tangga, hingga sistem pemantauan berbasis sensor. Sistem tertanam merupakan sistem komputer yang dirancang untuk menjalankan fungsi tertentu dengan memanfaatkan perangkat keras dan perangkat lunak yang saling terintegrasi. Salah satu komponen utama dalam sistem tersebut adalah mikrokontroler yang berfungsi sebagai pusat pengendali untuk menerima masukan, mengolah data, dan menghasilkan keluaran sesuai dengan program yang dijalankan. Oleh karena itu, pemahaman mengenai mikrokontroler dan cara pemrogramannya menjadi salah satu dasar penting dalam mempelajari sistem tertanam.

Dalam penerapannya, penggunaan mikrokontroler tidak terlepas dari beberapa permasalahan, seperti pengaturan komunikasi antara perangkat keras dan perangkat lunak, pembacaan sinyal masukan yang tidak stabil, serta pengolahan data dari sensor. Sebagai contoh, pembacaan tombol dapat menghasilkan perubahan sinyal yang tidak konsisten akibat kondisi floating maupun fenomena bouncing pada kontak mekanis. Selain itu, tidak semua informasi dari lingkungan dapat direpresentasikan hanya melalui kondisi digital HIGH dan LOW, sehingga diperlukan kemampuan untuk membaca sinyal analog menggunakan Analog-to-Digital Converter (ADC). Permasalahan tersebut menunjukkan bahwa pemahaman terhadap konfigurasi GPIO, penggunaan resistor pull-up dan pull-down, teknik debouncing, serta pembacaan ADC sangat diperlukan agar mikrokontroler dapat bekerja secara tepat dan sesuai dengan kebutuhan sistem.

Berdasarkan hal tersebut, praktikum Modul 1 dilaksanakan untuk memberikan pemahaman dasar mengenai pengenalan mikrokontroler dan penerapannya dalam pemrograman sistem tertanam. Praktikum ini menggunakan dua jenis mikrokontroler, yaitu STM32 Blackpill dan ESP32, dengan memanfaatkan PlatformIO pada Visual Studio Code sebagai lingkungan pengembangan. Selain mempelajari proses konfigurasi dan pemrograman menggunakan framework Arduino dan ESP-IDF, praktikan juga melakukan beberapa percobaan, mulai dari mengendalikan LED melalui program Blink, membaca kondisi tombol menggunakan konfigurasi pull-up dan pull-down eksternal maupun internal, menerapkan debouncing untuk mengatasi ketidakstabilan sinyal tombol, hingga membaca intensitas cahaya menggunakan sensor Light Dependent Resistor (LDR) melalui ADC. Melalui rangkaian percobaan tersebut, praktikan diharapkan dapat memahami hubungan antara perangkat keras dan perangkat lunak serta memperoleh keterampilan dasar dalam mengembangkan sistem berbasis mikrokontroler yang dapat diterapkan pada berbagai teknologi, khususnya perangkat IoT dan sistem otomasi.

1.2 Dasar Teori

1.2.1 Sistem Tertanam dan Mikrokontroler

Sistem tertanam (*embedded system*) merupakan sistem komputer yang dirancang untuk menjalankan fungsi tertentu dan biasanya menjadi bagian dari suatu perangkat elektronik. Sistem ini menggabungkan perangkat keras (*hardware*) dan perangkat lunak (*software*) agar dapat bekerja secara terintegrasi sesuai dengan kebutuhan. Salah satu komponen utama dalam sistem tertanam adalah mikrokontroler, yaitu sebuah chip yang di dalamnya terdapat unit pemroses, memori, serta berbagai *peripheral* yang digunakan untuk mengendalikan perangkat eksternal. Mikrokontroler bekerja dengan

menjalankan instruksi program untuk menerima masukan, mengolah data, dan menghasilkan keluaran. Contoh penerapannya dapat ditemukan pada perangkat IoT, sistem otomasi, sistem pemantauan lingkungan, dan berbagai perangkat elektronik yang membutuhkan pengendalian secara otomatis.

1.2.2 STM32 Blackpill dan ESP32

STM32 Blackpill merupakan salah satu *development board* yang menggunakan mikrokontroler dengan arsitektur ARM Cortex-M4 32-bit. Mikrokontroler ini memiliki kemampuan pemrosesan hingga frekuensi 84 MHz, memori Flash sebesar 256 KB, serta RAM sebesar 64 KB. STM32 Blackpill menyediakan sejumlah pin GPIO yang dapat digunakan sebagai masukan maupun keluaran untuk berinteraksi dengan komponen elektronik. Dalam proses pemrogramannya, board ini dapat menggunakan framework Arduino melalui STM32duino. Namun, proses pengunggahan program (*flashing*) membutuhkan perangkat tambahan berupa ST-Link yang berkomunikasi melalui antarmuka Serial Wire Debug (SWD).

ESP32 merupakan mikrokontroler 32-bit yang memiliki dua inti pemrosesan (*dual-core*) dengan arsitektur Xtensa LX6. Salah satu keunggulan ESP32 adalah tersedianya modul Wi-Fi dan Bluetooth yang sudah terintegrasi, sehingga banyak digunakan dalam pengembangan perangkat Internet of Things (IoT). Selain itu, ESP32 memiliki berbagai *peripheral*, seperti GPIO dan ADC, yang memungkinkan mikrokontroler berinteraksi dengan berbagai komponen dan sensor. Berbeda dengan STM32 Blackpill, ESP32 DevKit umumnya sudah dilengkapi dengan antarmuka USB-to-Serial sehingga dapat dihubungkan langsung ke komputer untuk proses pemrograman.

1.2.3 PlatformIO dan Framework Pemrograman

PlatformIO merupakan sebuah ekosistem pengembangan yang digunakan untuk membuat, mengelola, mengompilasi, dan mengunggah program pada berbagai jenis mikrokontroler. PlatformIO dapat diintegrasikan dengan Visual Studio Code sehingga proses pengembangan program dapat dilakukan dalam satu lingkungan kerja. Dalam sebuah proyek PlatformIO, terdapat beberapa bagian penting, salah satunya adalah berkas `platformio.ini` yang digunakan untuk mengatur jenis board, platform, framework, dan metode pengunggahan program. Selain itu, folder `src` digunakan untuk menyimpan kode program utama. Penggunaan PlatformIO mempermudah praktikan dalam mengembangkan program karena mendukung berbagai jenis mikrokontroler dan framework dalam satu lingkungan pengembangan.

Framework merupakan kumpulan pustaka dan fungsi yang digunakan untuk mempermudah proses pemrograman perangkat keras. Pada praktikum ini digunakan dua framework, yaitu Arduino dan ESP-IDF. Framework Arduino menyediakan fungsi yang lebih sederhana, seperti `pinMode()`, `digitalWrite()`, dan `delay()`, sehingga lebih mudah dipahami oleh pemula. Sementara itu, ESP-IDF (*Espressif IoT Development Framework*) merupakan framework resmi dari Espressif yang memberikan akses lebih luas terhadap fitur internal ESP32. ESP-IDF menggunakan struktur program dengan fungsi `app_main()` sebagai titik awal eksekusi dan dibangun di atas FreeRTOS. Salah satu keunggulannya adalah kemampuan dalam mengelola beberapa tugas (*task*) secara terstruktur, sehingga sesuai untuk pengembangan sistem tertanam yang lebih kompleks.

1.2.4 General Purpose Input/Output (GPIO)

General Purpose Input/Output (GPIO) merupakan pin pada mikrokontroler yang dapat dikonfigurasi untuk menerima masukan maupun menghasilkan keluaran digital. GPIO memiliki dua mode penggunaan utama, yaitu digital input dan digital output. Pada mode digital input, pin digunakan untuk membaca kondisi sinyal dari perangkat eksternal, seperti tombol atau saklar. Nilai yang dibaca umumnya berupa HIGH atau LOW yang merepresentasikan kondisi logika digital. Sementara itu, pada mode digital output, pin digunakan untuk mengendalikan perangkat eksternal, seperti LED, dengan memberikan keluaran berupa level tegangan tertentu. Pengaturan mode GPIO dilakukan melalui program menggunakan fungsi tertentu sesuai dengan framework yang digunakan. Pada Arduino, contohnya adalah fungsi `pinMode()` untuk menentukan mode pin dan `digitalWrite()` untuk mengatur kondisi keluarannya.

1.2.5 Pull-up dan Pull-down Resistor

Pada rangkaian digital, pin input mikrokontroler membutuhkan kondisi logika yang jelas agar dapat membaca sinyal dengan benar. Apabila sebuah pin input tidak terhubung dengan jalur tegangan HIGH maupun LOW, kondisi tersebut disebut *floating*. Kondisi floating menyebabkan nilai pembacaan pin menjadi tidak pasti karena dapat dipengaruhi oleh gangguan listrik atau *noise* dari lingkungan sekitar. Salah satu cara untuk mengatasi permasalahan tersebut adalah menggunakan resistor pull-up atau pull-down.

Pull-up resistor merupakan resistor yang menghubungkan pin input dengan sumber tegangan positif (VCC), sehingga ketika tombol tidak ditekan, pin akan berada pada kondisi HIGH. Ketika tombol ditekan dan terhubung ke GND, kondisi pin berubah menjadi LOW. Sebaliknya, pull-down resistor menghubungkan pin input dengan GND sehingga kondisi awal pin adalah LOW. Ketika tombol ditekan dan terhubung ke VCC, kondisi pin berubah menjadi HIGH. Kedua konfigurasi tersebut dapat diterapkan menggunakan resistor eksternal maupun resistor internal yang telah tersedia pada mikrokontroler. Pada ESP32, konfigurasi internal dapat diaktifkan melalui fungsi `INPUT_PULLUP` dan `INPUT_PULLDOWN` pada framework Arduino.

1.2.6 Debouncing

Debouncing merupakan teknik yang digunakan untuk mengatasi perubahan sinyal yang tidak stabil akibat karakteristik mekanis pada tombol atau saklar. Ketika sebuah tombol ditekan atau dilepaskan, kontak logam di dalamnya tidak selalu langsung terhubung atau terputus secara sempurna. Kontak tersebut dapat mengalami getaran mekanis dalam waktu singkat yang menyebabkan sinyal mengalami perubahan HIGH dan LOW secara berulang. Fenomena ini dikenal sebagai *bouncing*. Apabila tidak ditangani, mikrokontroler dapat membaca satu kali tekanan tombol sebagai beberapa kali masukan. Hal tersebut dapat menyebabkan kesalahan dalam penghitungan maupun pengendalian sistem. Salah satu metode untuk mengatasinya adalah *software debouncing*, yaitu dengan memberikan waktu tunggu tertentu untuk memastikan kondisi sinyal sudah stabil sebelum dianggap sebagai masukan yang valid. Dengan demikian, setiap tekanan tombol dapat dikenali dengan lebih akurat.

1.2.7 Analog-to-Digital Converter (ADC)

Analog-to-Digital Converter (ADC) merupakan komponen internal mikrokontroler yang berfungsi mengubah sinyal analog menjadi data digital agar dapat diproses oleh program. Berbeda dengan sinyal digital yang hanya memiliki dua kondisi logika, sinyal analog memiliki nilai tegangan yang dapat berubah secara kontinu dalam rentang tertentu. ADC bekerja dengan melakukan proses pengambilan sampel dan mengubah nilai tegangan tersebut menjadi representasi angka digital berdasarkan resolusi yang dimiliki. Pada ESP32, ADC memiliki resolusi 12-bit dengan rentang nilai pembacaan dari 0 hingga 4095. Nilai tersebut menunjukkan hasil konversi tegangan analog yang diterima oleh pin ADC. Semakin besar nilai resolusi ADC, semakin banyak kemungkinan tingkat nilai digital yang dapat digunakan untuk merepresentasikan sinyal analog. Dalam pemrograman Arduino, pembacaan ADC pada ESP32 dapat dilakukan menggunakan fungsi `analogRead()`.

1.2.8 Light Dependent Resistor (LDR)

Light Dependent Resistor (LDR) merupakan salah satu jenis sensor cahaya yang memiliki nilai hambatan listrik yang dipengaruhi oleh intensitas cahaya yang diterima. Ketika intensitas cahaya meningkat, nilai hambatan LDR cenderung menurun. Sebaliknya, ketika intensitas cahaya berkurang, nilai hambatannya akan meningkat. Perubahan hambatan tersebut dapat dimanfaatkan untuk mengetahui kondisi pencahayaan di lingkungan sekitar. Agar perubahan hambatan dapat dibaca oleh mikrokontroler, LDR biasanya dirangkai bersama resistor tetap sebagai pembagi tegangan (*voltage divider*). Rangkaian ini menghasilkan tegangan keluaran yang berubah sesuai dengan perubahan nilai hambatan LDR. Tegangan tersebut kemudian dihubungkan ke pin ADC mikrokontroler untuk dikonversi menjadi nilai digital. Dengan prinsip tersebut, ESP32 dapat digunakan untuk membaca perubahan intensitas cahaya dan mengolahnya sebagai informasi masukan dalam suatu sistem tertanam.

2 Tugas Pendahuluan

Bagian ini berisi jawaban dari tugas pendahuluan yang telah dikerjakan sebelum pelaksanaan praktikum. Setiap jawaban disertai penjelasan mengenai konsep dan prinsip kerja yang berkaitan dengan materi pada Modul 1, yaitu pengantar mikrokontroler dan dasar pemrograman sistem tertanam.

1. **Jelaskan perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.**

   Arduino dan ESP-IDF merupakan dua framework yang dapat digunakan untuk memprogram mikrokontroler, khususnya ESP32. Perbedaan utama keduanya terletak pada tingkat abstraksi dan cara pengelolaan program. Framework Arduino memiliki tingkat abstraksi yang lebih tinggi sehingga penggunaannya relatif sederhana dan mudah dipahami, terutama bagi pemula. Arduino menyediakan fungsi-fungsi siap pakai seperti `pinMode()`, `digitalWrite()`, `digitalRead()`, dan `delay()`. Struktur programnya juga lebih sederhana karena menggunakan dua fungsi utama, yaitu `setup()` yang dijalankan satu kali saat program dimulai dan `loop()` yang dijalankan secara berulang.

   Sementara itu, ESP-IDF (*Espressif IoT Development Framework*) merupakan framework resmi dari Espressif yang dirancang untuk memberikan akses lebih luas terhadap fitur internal ESP32. ESP-IDF menggunakan struktur program berbasis *task* dan *component*, dengan fungsi

`app_main()` sebagai titik awal eksekusi program. Framework ini juga dibangun di atas FreeRTOS sehingga memungkinkan pengelolaan beberapa tugas secara bersamaan. Kelebihan Arduino adalah proses pemrogramannya lebih sederhana, memiliki banyak pustaka, dan cocok untuk pembuatan prototipe maupun aplikasi sederhana. Namun, tingkat abstraksinya yang tinggi membuat beberapa konfigurasi perangkat keras menjadi kurang fleksibel. Sebaliknya, ESP-IDF memiliki kelebihan berupa kontrol perangkat keras yang lebih mendalam, pengelolaan task yang lebih fleksibel, serta akses langsung terhadap berbagai fitur ESP32. Kekurangannya adalah struktur program lebih kompleks dan membutuhkan pemahaman yang lebih mendalam mengenai sistem tertanam. Dengan demikian, Arduino lebih sesuai untuk pembelajaran awal dan pengembangan sederhana, sedangkan ESP-IDF lebih sesuai untuk pengembangan aplikasi yang membutuhkan kontrol dan pengelolaan sistem secara lebih kompleks.

2. **Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk diflash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.**

   Perbedaan tersebut disebabkan oleh adanya perbedaan antarmuka komunikasi dan fasilitas pemrograman yang tersedia pada masing-masing board. STM32 Blackpill tidak memiliki chip USB-to-Serial maupun programmer yang terintegrasi pada board-nya. Oleh karena itu, proses pengunggahan program memerlukan perangkat tambahan berupa ST-Link yang berfungsi sebagai programmer sekaligus debugger. ST-Link berkomunikasi dengan mikrokontroler STM32 melalui antarmuka SWD (*Serial Wire Debug*), menggunakan jalur SWDIO sebagai jalur data dan SWCLK sebagai jalur clock. Melalui komunikasi tersebut, program dapat ditulis ke dalam memori Flash mikrokontroler.

   Berbeda dengan STM32 Blackpill, board ESP32 DevKit umumnya sudah dilengkapi dengan chip USB-to-Serial, seperti CP2102 atau CH340. Chip tersebut berfungsi sebagai penghubung antara komunikasi USB pada komputer dengan antarmuka UART pada mikrokontroler ESP32. Ketika ESP32 dihubungkan menggunakan kabel USB, data program dari komputer dapat diteruskan melalui USB-to-Serial menuju ESP32 untuk proses flashing. Dengan demikian, ESP32 tidak memerlukan programmer eksternal seperti ST-Link karena fasilitas komunikasi untuk proses pengunggahan program sudah tersedia pada board. Perlu diperhatikan bahwa USB-to-Serial berfungsi sebagai penghubung komunikasi, sedangkan proses flashing tetap melibatkan mekanisme bootloader yang tersedia pada ESP32.

3. **Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.**

   Pull-up dan pull-down merupakan konfigurasi resistor yang digunakan untuk memastikan pin input mikrokontroler memiliki kondisi logika yang pasti ketika tombol tidak ditekan. Hal ini diperlukan karena pin input yang tidak terhubung dengan sumber tegangan maupun ground dapat mengalami kondisi *floating*, yaitu keadaan ketika nilai logika pin tidak dapat ditentukan secara konsisten akibat pengaruh gangguan listrik atau noise.

   Pada konfigurasi pull-up, resistor dihubungkan antara pin GPIO dengan sumber tegangan positif (VCC), sedangkan tombol dihubungkan antara pin GPIO dengan GND. Ketika tombol tidak ditekan, arus dari VCC melalui resistor akan mempertahankan kondisi pin pada logika HIGH. Sebaliknya, ketika tombol ditekan, pin GPIO terhubung langsung ke GND sehingga kondisi lo-

gikanya berubah menjadi LOW. Pada konfigurasi pull-down, resistor dihubungkan antara pin GPIO dengan GND, sedangkan tombol menghubungkan pin GPIO dengan VCC. Akibatnya, ketika tombol tidak ditekan, pin berada pada kondisi LOW, sedangkan ketika tombol ditekan, pin berubah menjadi HIGH. Perbedaan kedua konfigurasi tersebut dapat dilihat pada Tabel berikut.

Tabel 1: Perbandingan kondisi logika pull-up dan pull-down

Kondisi Tombol	Pull-up	Pull-down
Tidak ditekan	HIGH	LOW
Ditekan	LOW	HIGH

Kedua konfigurasi tersebut dapat diterapkan menggunakan resistor eksternal maupun resistor internal yang sudah tersedia pada mikrokontroler. Pada ESP32 dengan framework Arduino, konfigurasi pull-up internal dapat diaktifkan menggunakan `INPUT_PULLUP`, sedangkan pull-down internal menggunakan `INPUT_PULLDOWN`.

4. **Jelaskan fenomena bouncing pada tombol mekanik, mengapa hal ini bisa terjadi, dan sebutkan minimal dua pendekatan untuk mengatasinya.**

   Bouncing merupakan fenomena perubahan sinyal digital yang terjadi secara berulang dalam waktu singkat ketika tombol mekanik ditekan atau dilepaskan. Fenomena ini disebabkan oleh adanya getaran dan pantulan mekanis pada kontak logam di dalam tombol. Ketika kedua kontak tersebut bersentuhan, keduanya tidak selalu langsung terhubung secara stabil, melainkan dapat mengalami beberapa kali kontak dan pemutusan sebelum akhirnya berada pada kondisi yang stabil. Akibatnya, mikrokontroler dapat membaca satu kali tekanan tombol sebagai beberapa perubahan logika HIGH dan LOW. Jika tidak ditangani, kondisi ini dapat menyebabkan kesalahan pembacaan input, seperti penghitung yang bertambah lebih dari satu kali akibat satu tekanan tombol.

   Terdapat beberapa pendekatan yang dapat digunakan untuk mengatasi bouncing. Pertama, *software debouncing*, yaitu metode yang dilakukan melalui program dengan memberikan jeda waktu tertentu untuk memastikan kondisi tombol sudah stabil sebelum dianggap sebagai input yang valid. Salah satu contohnya adalah menggunakan fungsi `millis()` pada Arduino untuk menghitung waktu sejak perubahan sinyal terakhir. Kedua, *hardware debouncing*, yaitu metode yang menggunakan komponen elektronik tambahan, seperti rangkaian resistor dan kapasitor (RC), untuk meredam perubahan tegangan yang terjadi secara cepat. Selain itu, rangkaian Schmitt Trigger juga dapat digunakan untuk menghasilkan perubahan logika yang lebih stabil dengan memanfaatkan karakteristik ambang tegangan yang berbeda saat sinyal naik dan turun. Dengan menerapkan metode tersebut, pembacaan tombol oleh mikrokontroler dapat menjadi lebih akurat dan mengurangi kemungkinan terjadinya pembacaan ganda.

5. **Bagaimana ADC (Analog-to-Digital Converter) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.**

   ADC (*Analog-to-Digital Converter*) merupakan komponen yang berfungsi mengubah sinyal analog berupa tegangan listrik menjadi nilai digital agar dapat diproses oleh mikrokontroler. Proses konversi ini dilakukan dengan mengambil sampel tegangan analog, kemudian menentukan nilai digital yang paling mendekati tegangan tersebut berdasarkan rentang tegangan referensi dan

resolusi ADC. Dengan demikian, perubahan tegangan yang bersifat kontinu dapat direpresentasikan menjadi angka digital yang dapat dibaca dan diolah oleh program.

Resolusi ADC ditentukan oleh jumlah bit yang digunakan dalam proses konversi. Semakin besar jumlah bit, semakin banyak tingkat nilai digital yang dapat dihasilkan sehingga perubahan tegangan dapat direpresentasikan dengan lebih detail. Jumlah tingkat nilai digital dapat dihitung menggunakan persamaan:

$$N = 2^n \tag{1}$$

dengan N merupakan jumlah tingkat nilai digital dan n merupakan resolusi ADC dalam satuan bit. Sebagai contoh, ADC dengan resolusi 8-bit memiliki $2^8 = 256$ tingkat nilai digital, sedangkan ADC 12-bit memiliki $2^{12} = 4096$ tingkat nilai digital. Pada ESP32, ADC memiliki resolusi 12-bit dengan rentang pembacaan nilai 0 hingga 4095.

Ukuran perubahan tegangan terkecil yang dapat direpresentasikan secara ideal dapat dihitung menggunakan persamaan:

$$\Delta V = \frac{V_{\text{ref}}}{2^n} \tag{2}$$

dengan ΔV merupakan ukuran satu tingkat kuantisasi, V_{ref} merupakan rentang tegangan referensi, dan n merupakan resolusi ADC. Apabila menggunakan rentang tegangan 0–3,3 V dan resolusi 12-bit, ukuran satu tingkat kuantisasi secara ideal adalah:

$$\Delta V = \frac{3,3}{4096} \approx 0,000806 \text{ V} \tag{3}$$

Hasil tersebut menunjukkan bahwa setiap kenaikan satu tingkat nilai digital mewakili perubahan tegangan sekitar 0,806 mV. Dengan demikian, semakin tinggi resolusi ADC, semakin kecil perubahan tegangan yang dapat dibedakan. Namun, resolusi yang tinggi tidak selalu menjamin akurasi pengukuran karena hasil pembacaan juga dipengaruhi oleh noise, kestabilan tegangan referensi, dan karakteristik ADC itu sendiri.

6. **Lakukan set-up untuk tools: VSCode, ekstensi PlatformIO (dengan framework ESP-IDF dan STM32), dan ekstensi Serial Plotter pada laptop kalian. Berikan screenshot bahwa laptop kalian sudah ter-set-up dengan baik.**

